Stanford Compilers

Lecture 1

- Interpreters → JS, Python → run program → Translates one line at a time
- Compiles translates → Low level: C, C++ → Converts entire program first before running it

1. Lexical — Words Recognize
2. Parsing — Series Identify
3. Semantic — Analyzes Series
4. Optimization — Editing
5. Code Generation — Translation

(1) → Lexical Analyzer divides into words/tokens
- tokens like "if", "x", "==", etc.

(2) Parsing → Diagramming sentences (diagram is a tree)

(3) Semantic → Capture "meaning"
- Rather "limited" mainly to catch inconsistencies
- Define strict rules instead
  - Make sure all "grammar" is correct

(4) Optimization → Similar to MX bypassing e.g. where we minimize ready time to keep it in short-term memory.

?? $x = y * 0$, can be simplified to $x = 0$ in "most cases". What if $y = NaN$ or $y = null$ b/c it threw an exception??

Then we get NaN * 0 = NaN

Code Gen typically produces assembly code
- An example of alternative would be an IR (LLVM e.g.)
  - IR, Low levels can expose feature hidden by higher levels
  - But we don't want it to elimate high-level meaning.

* But we have many trade-offs in language design.

Lecture 2 → Overview of COOL

- Success of language is driven by economics + adoption not how "good" the language is.

e.g. JS "exploded" b/c of needy websites interactive.
→ So there are different need for example

e.g.

Abstraction = "Selective Ignorance" and hiding important info. Done in the form of f(x), modules, classes etc.

Types
- Help enforce abstraction
- Optimize code
- Identify frequent errors

Various form of parameterization

Connection to Software Analysis

Reusing code + Software Structures + Inheritance for example

Language design → Popular languages stick around instead may new special-purpose languages

COOL → classroom Object Oriented Language
- Give just a taste of modern copilers Abstraction, static typing etc.

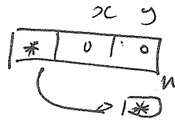
→ MIPS

Cool programs → set of class definitions
- All cool code lives inside classes which is only attributes and methods. (Hence OOP)

e.g. "new Point" makes an ~~class~~ object of class Point

~~this~~ self ✓

methods are global



→ contain slot pointing to code.

→ do move Point()

Types:
Int, Bool, String, Object, Inheritance are dispatched thru dynamic types

cool GC

new invokes mem, GC cleans up when no longer reachable

Lecture 3

Video → Lexical Analysis

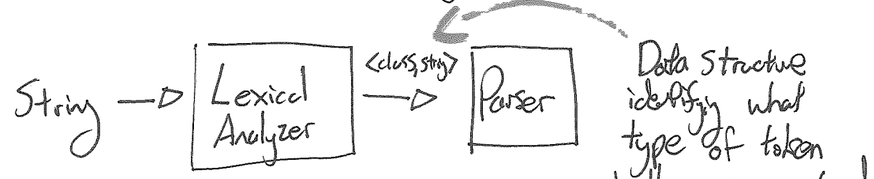
Goal is to divide code into Lexical units — including keywords such as "if" and operators e.g.

Break into tokens → Identifier, Keyword, "(", ")" e.t.c.

- Identifier
- Integer
- Keyword
- Whitespace

} Token Class

Classify program substring according to role/token class



e.g. foo = 42

<Id, "foo"> <Op, "="> <"Int", "42">

↑ Lexemes

↳ chunks it up

Identifier → A valid name chosen by programmer
↳ e.g. "counter" or "banana"

So Lexical Analyzer must:

1) Recognize substrings corresponding to tokens
2) Identify token class of each lexeme

Examples: Lookahead [minimize lookahead]

↳ Unbound lookahead (there's no "eof" in sight)

↳ Helps us understand what role current symbol plays in language

Reg Expressions

$$\begin{bmatrix} R = \varepsilon \\ \mid \text{'c'} \\ \mid R+R \\ \mid RR \\ \mid R^* \end{bmatrix} \quad c \in \Sigma$$

The smallest set of expressions over Σ alphabet including

e.g. the alphabet can be $\Sigma = \{0,1\}$

$1^* = \bigcup_{i \geq 0} 1^i = "" + 1 + 11 + 111 + \cdots + \underbrace{1\cdots1}_{i} + \cdots$ = all strings of 1's

Regular expressions specify regular languages which indeed or is simply defined as a set of strings.

2/ Eval 1-char strings, 3/ Union, concat, iterate

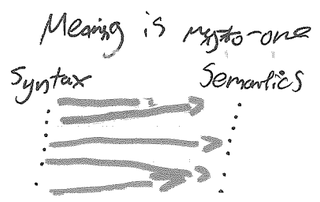
Formal Languages

Σ is a set of characters (an alphabet) while a language over Σ is a set of strings of characters drawn from Σ.

Alphabet = English chars
Language = Rules with English Sentences

$L : Exp \rightarrow$ Sets Strings

Meaning is many-to-one



$AA^* \equiv A^+$

e.g. digit digit*

Set of digits, the expression starts with a digit and then is followed by a large number of digits

letter = [a-z A-Z]

letter (letter + digit)*

Lecture 4

A method to go about actually parsing a string into its different parts

Adapt regular expressions to this problem accordingly

#1. Write rexp for lexemes of each token class

e.g. Number = digit+
Keyword = "if" + "else" + ...

#2. Construct R which matches the lexemes for all tokens:

R = keyword + ...

#3 → #5 => A for loop where we check based on a string whether the match is minimum ∈ R for the lexer match

Remove input ambiguity to R ~> Maximal munch / longest-match to tokenize

=> we want the longer one, == e.g. the longer double == not the single =

which token to use if it's longer

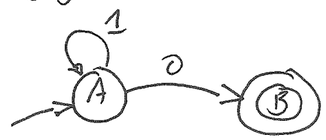
e.g. if ∈ L(keywords) L(Identifiers) <- We need a priority

No rule matches => Error regex placed last

Finite Automata = implementation of regular expressions

transition $s_1 \xrightarrow{a} s_2$, In state s_1 on input a go to state s_2

Language of a FA ≡ Set of accepted strings

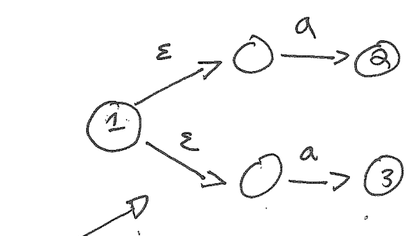


This is the final ^ state (accepting)

e.g. 100 rejects as ready final zero doesn't give exit accepting route

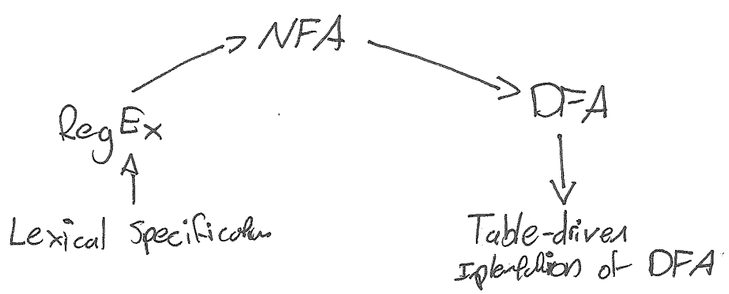
Deterministic Finite Automata (DFA) => one transition per input per state
- No ε-moves

Meanwhile a nondeterministic Finite Automaton
↳ Can have multiple transitions for one input in a given state
↳ Can have ε-moves

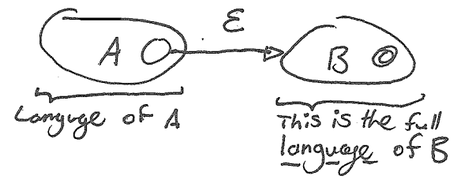


So we can write out the states such that the input still only has one state for transition from ε to a to 2 and thus the only difference is whether or not it has an ε move [absorbed out]

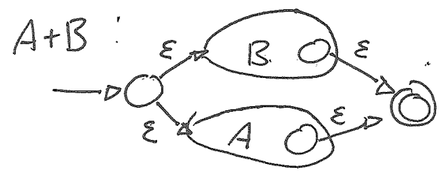
NFA can choose state
↳ Machine accepts input if it ends at a state, if there is a set of states that can be chosen, that is described as an accepts state
↳ But DFAs are faster to execute



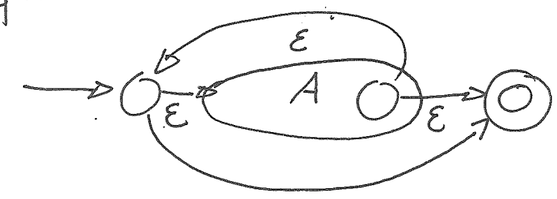
AB



A+B



A*



NFA_to_DFA ← Always one state

S is said state present in $s \in S$, where S is the set of all states

a(x) → where can we go on from character "a"

ε-clos(x) → All states reachable from x using only epsilon moves

b/c NFA can be in many states at any time
↳ We have N states, so $|S| \leq N$ leads to $2^N - 1$ finite set of possible configs

NFA

States S
Start $s \in S$
Final $F \subseteq S$

$a(X) = \{y \mid x \in X \wedge x \xrightarrow{a} y\}$ ← Function to position from x to y on input a

This is all the states you can reach on input a → If I start from any state inside the set X, where can I end up after reading input character "a"??

ε-clos ← Starting from state "s", what states can I reach only using epsilon ε transitions??

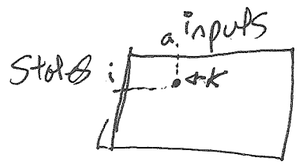
DFA

States subset of S
Start $\varepsilon\text{-clos}(s)$
final $\{X \mid X \cap F \neq \emptyset\}$

$X \xrightarrow{a} Y$ if $Y = \varepsilon\text{-clos}(a(X))$ ← we have "a" transitions from X to some Y states. Then once we're at Y final states, we can make ε choices of where to go next, hence, the ε-clos choice.

A DFA can be implemented by a 2D table T

$S_i \xrightarrow{a} S_k$ defines $T[i,a] = k$



Lecture 5

Intro to Parsing

What can regular ~~expressions~~ languages express??
↳ Parity but can't count such as: $(^i)^i$

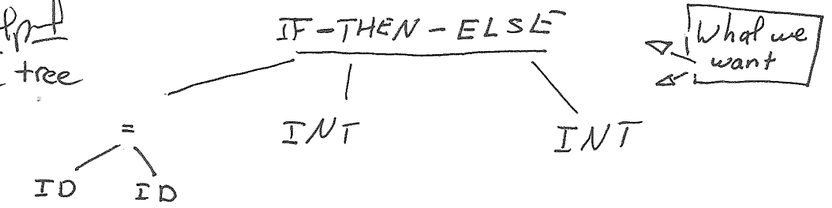
It can:

mod k

Input: Sequence of tokens from lexer
output: parse tree of the program

e.g. IF ID = ID THEN INT ELSE INT FI

output parse tree



* May be implicit, often parsing gets handled seperately though

Parser must distinguish b/w valid and invalid strings of tokens.

We need

- Language to describe valid strings of tokens
- Need to distinguish b/w valid and invalid tokens
  - Naturally have recursive structure

All these languages have
Context Free Grammars (CFG)

- Terminals, T → Symbols/tokens in final input
- Non-Terminals N → Grammar that doesn't appear in source code
- Start S, (S ∈ N) → A N as the start program
- Set of productions, $X \to Y_1 \dots Y_N$
  ↳ Grammar rules

Productions can read as rules

1. Begin w start symbol S
2. Replace non-terminal X by right-hand side of some production $X \to Y_1 \dots Y_n$
3. Repeat (2) until we have no non-terminals

So one step of context free derivation:

$$X_1 \dots X_i \underline{X} X_{i+1} \dots X_N \to X_1 \dots X_i \underline{Y_1 \dots Y_K} X_{i+1} \dots X_N$$

b/c of the production rules:

$$X \to Y_1 \dots Y_K$$

So $\alpha_0 \xrightarrow{*} \alpha_n$ (in ≥ 0 steps)

we have some start symbol to final state in n steps

↳ When G is a context-free grammar with start symbol S, then langage L(G) of G is:

$$\{a_1 \dots a_n \mid \forall i,\ a_i \in T \wedge S \xrightarrow{*} a_1 \dots a_n\}$$

Starting from S after zero or more grammar rules, we can derive the string $a_1 \dots a_n$

L(G) is the collection of all valid string/programs the grammar can generate

Context free for the parser doesn't require you to see what's present before
↳ This is the semantic analyzer which walks through the parse tree and maintains context.

→ Terminals are permanent once generated
↳ which can be replaced and are tokens/parts of the language

Derivation is a sequence of productions

$S \to \dots \to \dots$

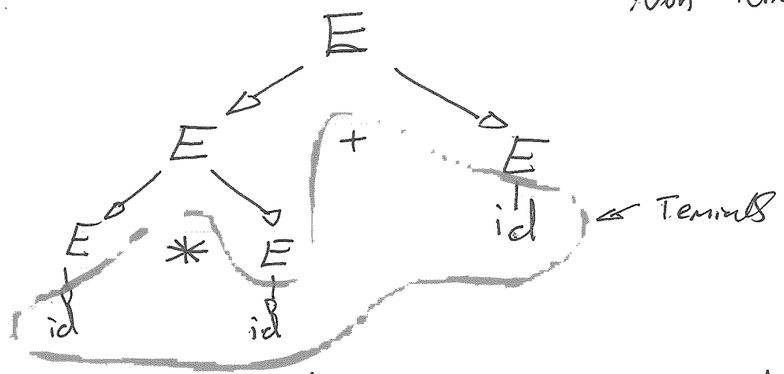
A derivation can be drawn as a tree

e.g. $X \to Y_1 \dots Y_n$ and children $Y_1 \dots Y_n$ under node X

Such that:

Start Symbol is the tree's root

Parse Tree Example



So an in order traversal gives: id * id + id
and all the leaves have only Terminals. All the interior nodes are Non-terminals

→ All parse trees have right-most & left-most derivs, which give some parse tree

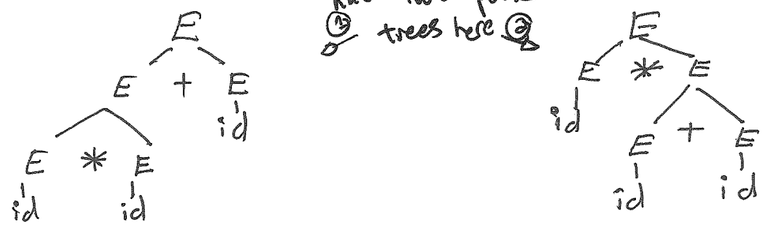
☆ We are not just interest in $S \in L(G)$ but we also need a parse tree for S.

Ambiguity ↝ Has more than one parse tree for some string. Bad!

e.g. ~~An~~

id * id + id

has two parse trees here



How to handle ambiguity

↳ Rewrite grammar unambiguously.

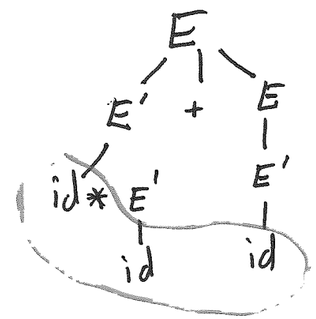
e.g. rewrite as

→ Where each subscript regex is one desired tree

$E \to E' + E \mid E'$

$E' \to id * E' \mid id \mid (E) * E' \mid (E)$

id * id + id



Notice how this is the only tree setup that can be generated from the id * id + id string, making it unambiguous

---

$\varepsilon, a, b, aa, ab, \ldots$ with OG $S \to Ss \mid a \mid b \mid \varepsilon$

So it can give diff parse trees

$S \to Sa \mid Sb \mid \varepsilon$

To make things cleaner we can add precedence declarations, such as:

%left +

%left *

Both correspond to associative disambiguity

Lecture 6

↳ Continuing with Parsing

Compiler Purpose:

↳ Detect non-valid progs
↳ Translate valid ones

- Lexical
- Syntax
- Semantic

Types

~

- Panic Mode
- Error Productions

↳ Look for Synchronizing tokens

→ Skip ahead to next integer ~~before~~ and continue on from there.

E -> int | E+E | (E) | error int | (error)

How many variables/expressions that are normally defined

found an error

~~Common Mistakes~~ Specify know common mistakes in grammar

e.g. E->... | EE when Sxc instead of S*x

<=> Makes it ~~very~~ complicated

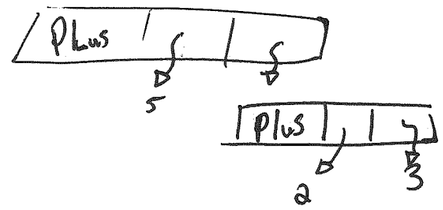
Error correction → Correct "nearby" programs → edit distance

↳ Nearby programs

Abstract Syntax Tree

Compiler needs a datastructure (Structural) representation of the program.

↳ AST like a parse tree w/o some of the details



Captures nesting structure but absents away from concrete syntax

Recursive Descent

Algorithm top-down

↳ Generate non-terminals until we reach a terminal and check that b/c we recursed down change non-terms to terminals until we have a valid output

Recursive Descent Algorithm

↳ Create boolean functions checking for a match
  ↳ Token Terminal
  ↳ n^{th} production of S
  ↳ All products of S

e.g. Notice how each term has its own function

$E \rightarrow T+E$

```
bool E2() {return T() && term(PLUS) && E();}
```

```
bool T(){
    TOKEN *save = next;
    return (next = save, T1())
        || (next = save, T2())
        || (next = save, T3());}
```

This all 'next' are the pointers telling us where in the string we are at.

Going into each terminal function

Init next to point to first token
→ Invoke E()

RDA Limitation can occur with backtracking in a succeeding terminal
↳ If it succeeds we can't backtrack to try a different production for X later
  ↳ Easy to implement by hand
  ↳ Difficult for NT where at most one production can succeed.

Left Recursion

A left-recursive grammar has non-terminal S such that: $S \rightarrow^{+} S\alpha$ for some α

Where the first thing we generate is a non-terminal signed S

↳ So instead rewrite with right-recursion

$S \rightarrow \beta S'$ and $S' \rightarrow \alpha S' | \varepsilon$

As a result the left and right recursions "balance" out as we go from left then to right

$S \rightarrow A\alpha | \delta$
$A \rightarrow S\beta$
} Still left-recursive b/c $S \rightarrow^{+} S\beta\alpha$ when you combine both left and right recursive sides

Lecture 7

- Predictive Parsers
  - Looks at next few tokens
  - No backtracking

This avoids picking a wrong rule as it instead looks at the next tokens before then choosing a rule.

- Predictive Posers accept LL(k) grammars
  - Left to right
  - Leftmost
  - k tokens lookahead

e.g. LL(1), we only see one ahead at see one choice of picking ahead

Edit the grammar to delay deciding on which production we're using. Wait and see.

$E \to TX$
$X \to + E \mid \varepsilon$ ← for the first one

Left-factor the grammar

LL(1) parsing table allows you to look ahead and know which grammar rule to use in $O(1)$ time.

- LL(1) is a 2D table
- LL(2) would be a 3D table ← Higher than that would require alternative means instead of longer tables.

A Stack records frontier of parse tree
- NT have yet to be expanded
- Top of Stack ↔ Leftmost pending terminal or non terminal

Reject on reaching error state
- Accept on end of input & empty stack

Example of using the ~~cache~~ stack to pop-off inputs and corresponding actions down the parsing tree.

First Sets

if $\alpha \to^* t\beta$
- α can derive a t in the first position, we say $t \in First(\alpha)$

if $A \to \alpha$ and $\alpha \to^* \varepsilon$ and $S \to^* \beta A t \delta$,

Imagine the parser stack has an A at the top
~~If~~ if A can't generate the next token, the parser can use $A \to \varepsilon$ to make A disappear.

Definition

$$First(X) = \{t \mid X \to^* t\alpha\} \cup \{\varepsilon \mid X \to^* \varepsilon\}$$

- If I begin parsing X, what is the first token I might see in the input??' → used as a shortcut.
  - If it's first it only considers rules that can start with 'int'.

also:
3) $First(\alpha) \subseteq First(X)$ if $X \to A_1 \ldots A_n \alpha$ ($A_1 \ldots A_n \to \varepsilon$)
2) $\varepsilon \in First(X)$
1) $First(t) = \{t\}$

All these disappear

This shows up in final string

e.g.

$E \to TX$ $\quad X \to +E \mid \varepsilon$

$T \to (E) \mid int\ Y$ $\quad Y \to^{*} T \mid \varepsilon$

Terminals are the same

First (+) = {+}
First (*) = {*}
⋮ {(}
{)}
{int}

First (E) ⊇ First(T)
First (T) = {(, int}
First (X) = {+, ε}
First (Y) = {*, ε}

However, we see that the first of T doesn't go to an ε, so the first of E has to always go to T.

↳ Thus First(E) = First(T)

Follow sets

Definition:

$Follow(X) = \{\epsilon \mid S \to^{*} \beta X \epsilon \delta\}$

if $X \to AB$ then $First(B) \subseteq Follow(A)$

↳ e.g. anything in First(B) will be in Follow set of A

And $Follow(X) \subseteq Follow(B)$

e.g. $S \to X\epsilon \to AB\epsilon$

$\$ \in Follow(S)$
$First(\beta) - \{\varepsilon\} \subseteq Follow(X)$
$Follow(A) \subseteq Follow(X)$, $\varepsilon \in First(\beta)$

Always just sets of terminals

Exercise

[Look anywhere E is used and see what's next, what follows].

$Follow(E) = \{\$,)\} = follow(X)$
$follow(T) = \{\$, +,)\} = Follow(Y)$

$Follow(X) \subseteq Follow(E)$
$Follow(E) \subseteq follow(X)$, so $Follow(X) = Follow(E)$
$Follow(E) = follow(X)$
$Follow(E) \subseteq Follow(T)$ ← b/c X can be ε

$Follow(Y) = Follow(T)$

→ Follow('(') = {(, int}
Follow(')') = {+, $,)}
Follow('+') = {(, int}
Follow('*') = {(, int}
Follow('int') = {*, +, $,)}

Now lets make an LL(1) Parsing Table
↳ call it table T for CFG G

	(	)	+	*	int	$
E	TX				TX	
T	(E)				int Y	
X	Error	ε	+E			ε
Y		ε	ε	*T		ε

S	a	b	$
		5	
		s_a	

Multiply defined entry!

If any entry is multiply defined, then G is not LL(1)
- not left factored
- left recursive
- ambiguous

} Not LL(1)

Bottom-up Parsing

↳ More general + just as efficient
→ Don't need left factored grammars, so can be natural instead

Reduce a string to the Start Symbol by inverting productions → Trace a rightmost derivation
↳ Traces a rightmost derivation in reverse

Builds from leaves of tree (typically terminals up to the root)

Shift reduce Parsing

↳ Undoing the most recent expansion that happened for undoing the rightmost derivation

→ There can't be any unfinished grammar work sitting to the right of the next reduction. B/c $\alpha\beta w$, so assume next reduction is by $X \to \beta$, so we are going to the w must be (only) terminals or

$$\alpha X w \to \alpha \beta w$$

So... Split String into two substrings
↳ Right Substring is not expanded
↳ Left Substring has terminal and non-terminals

$\alpha X w$ ← unexamined input

Left String can be implemented by a Stack
↳ Top of stack is

Shift-reduce → conflicts, where you can't reduce
Reduce-reduce → conflicts, where you can't reduce further down.

Shift and Reduce

e.g.
Cbaay|ijk => CbA|ijk

Lecture 8

Handles

Review

- Shift pushes terminal to the stack
- Reduce - pops 0+ symbols off the stack
  - pushes a non-terminal on the stack

When to Shift or reduce??

↳ Might be a mistake to blindly reduce, we can get "locked out"

Intuition: Only reduce if result can still be reduced to the start symbol

$S \rightarrow^* \alpha X w \rightarrow \alpha \beta w$

reduction

Then $\alpha\beta$ is a handle of $\alpha\beta w$

Handle: A reduction allowing further reduction back to the start symbol

↳ We only want to reduce at handles

(i.e. it is the next 'thing' a bottom-up parser should reduce)

☆ Handles appear only at the top of the stack, never inside

This is b/c bottom-up parsing parsing reverses a rightmost derivation, the newest piece is always the next one to reduce

Since it's next, it's always at the top of the stack

Handles are never to the left of the rightmost non-terminal

↳ shift-reduce moves are sufficient

Recognizing Handles

→ No efficient algo to recognize handles

→ But there are good heuristics to guessing handles → always guess correctly

- Set of viable prefixes, for any grammar, is a regExp

→ How to compile automaton accepting viable prefixes

item: Production (rule) with a "." on rhs

Shows us how far through the production the parser is currently at and how much its matched of the rule. (e.g. I ie matched entire rule)

☆ Lets us know how far into a production we are at. B/c the dot serves as the bookmark point

Stack has only bits + pieces of rhs of productions

So only contains prefixes which are the beginning of RHS but the entire RHS which can be reduced

↳ right-hand side

-> We don't always have the rhs on the stack, which are always prefixes

$T \to (E.)$, we are "working" on this production and have so far seen "(E" and hope to see ")".

$Prefix_{k+1} \dots Prefix_n$ exactly reduces to the missing part of α_k

e.g. consider the state of: (int *|int) which is a state of shift reduce parse

-> "("
-> "E"
-> "int *"

Prefix of rhs of $T \to (E)$, $E \to T$, $T \to int * T$

"Stack of items"

Recognize sequence of partial rh's of production which can reduce to part of missing suffix.

Recognizing Viable Prefixes (algorithm)

Only Two kinds of moves

3. $E \to \alpha . X\beta \to X \quad E \to \alpha X . \beta$

X can be "identified" as its own item as we shift, already on stack

4. $E \to \alpha . X\beta \xrightarrow{\varepsilon} X \to . \gamma$

X is a production with additional components.

Dot fully on right show it could be a handle to reduce

Valid Items

↳ States of Deterministic Automata are listed out as a "canonical collection of items"

$X \to \beta . \gamma$ is valid for $\alpha\beta$ prefix (specifically LR(0) item) if $S' \to^* \alpha X w \to \alpha\beta\gamma w$

↳ After parsing $\alpha\beta$ valid items are possible tops of stack of items.

SLR Parsing

-> LR(0) Shifts or reduces only current DFA state w/o looking ahead

↳ Makes decision using only parser state and ignores next input entirely

SLR ↳ Only reduce if next token is in the Follow(x)

LR(0)

Reduce by $X \to \beta$ if
- S contains $X \to \beta$

Shift if
- S contains $X \to \beta . \epsilon w$

LR(0) conflicts

1) Reduce/Reduce conflicts
↳ Two possible reduce items
2) Shift/reduce conflicts
↳ Reduce and shift item at same time

SLR → Improves on LR(0) so fewer states have conflicts
↳ But not all grammars are SLR

↳ But we can parse more grammars with "precedence declarations"
Instructions for resolving conflicts
precedence ≈ conflict resolution

- k is the # of lookahead

↳ DFA tells where it is in the grammar

SLR Parsing Example

e.g.

int * int $			
	2		Shift
	3	* not in Follow(T)	Shift
	11		Shift
	3	$ is in Follow(T) so it doesn't Shift as it's not in Follow(T)	red. T → int
	3		red. T → int * T
	4		
	5		red. E → T

Not in Follow(T) ⇒ don't reduce T

- Most of the work is repeated
  ↳ Remember state of automaton on each prefix

- Remember state of automaton on each prefix of stack
  ↳ Change stack to contain pairs ⟨Symbol, DFA State⟩ — Improvements

where $State_n$ is the final state of the DFA on $Sym_1 \dots Sym_n$ — Avoid extra work

Define $goto[i, A] = j$ if $State_i \xrightarrow{A} State_j$

↑ Transition function of DFA

- Shift
- Reduce
- Accept
- Error

} ⇒ New rules for "pair" "datastructure" on stack

LR(1) item builds lookahead into the items

LR(1) item = LR(0) item × lookahead

[T → .int * T, $] means that after seeing T → int * T, reduce if lookahead is $
↳ More accurate than just using follow set

e.g.

S → S o S .
S → S . a S

← We have a Shift-reduce conflict
Since follow(S) = {$, a}
Not LR(1)

Lecture 9

Intro to Semantic analysis

Idea: catch all remaining errors, the "front end"
↳ Last "front end" phase

- Compare this to parsing where we look to detect inputs with incorrectly formed parse trees.
  - ↳ Parsing can't catch some errors

★ Requirements depend on the language

Scope → portion of program where the identifier is accessible
↳ same identifier may reference different things in different parts of program

Dif scopes for same name don't overlap or nested scope
↳ static scope > dynamical scope

Most - closely nested identifiers
↳ e.g.
- class names
- Attribute names
- Method names

Much Semantic analysis ↳ Recursive descent of an AST

Symbol Tables

↳ We can use a stack and define a list of methods to push current "scope" onto the stack

⇝ Thing is class names can be used b/f defined
↳ So we instead perform two passes

Types

Types
↳ Set of values
↳ Set of operations on those values

int, +, -, ≥, <
strings concat

Function Pointer + Integer, doesn't make sense to add two integers

- Type checking just checks operations are used only with the correct types.
  - ↳ Machine can do anything but we need to enforce the intended interpretation of values.

Machine code is untyped e.g., statically, dynamically

Type Checking → Verifying fully typed programs
Type Inference → Process of filling in missing type info

inference rules

↳ If hypothesis is True => Conclusion is True

e.g.

$$[(e_1 : Int \wedge e_2 : Int) \Rightarrow e_1 + e_2 : Int]$$

If it is provable ... then (provable: numerator; then: which is the den)

$$\left[\frac{\vdash e_1 : Int \quad \vdash e_2 : Int}{\vdash e_1 + e_2 : Int}\right] \text{[Add]}$$

Sound when if

$\vdash e : T$ then e evaluates to a value of type T

Type checking proves facts that $e : T$?

new T produces an object of type T

$$\frac{}{\vdash \text{new } T : T} \text{[New]}$$

$$\frac{\vdash e : Bool}{\vdash \text{!}e : Bool} \text{[not]}, \quad \frac{\vdash e_1 : Bool \quad \vdash e_2 : T}{\vdash \text{while } e_1 \text{ loop } e_2 \text{ pool} : Object} \text{[Loop]}$$

$$\frac{x \text{ is a variable}}{\vdash x : ?} \text{[Var]}$$

Type environment gives types for free variables

↳ This is a function from objectIdentifiers to Types
↳ free if it's not defined within the expression

e.g. x is free while y is bound if defined

Let O be the function from objectIdentifiers to Types

$O \vdash e : T$ ↳ Assuming free variables have types by O it is provable e has type T.

e.g. $$\frac{O \vdash e_1 : Int \quad O \vdash e_2 \; Int}{O \vdash e_1 + e_2 : InT}$$, We can look at assumptions O for the types.

↙ Symbol Table

$O[T/x](y) = O(y)$ where $x \neq y$

e.g. $O \vdash e : T$, under type environment O, expression e has type T

O gets passed down AST as we add new definitions.

Define a relation $\leq$ on classes

$X \leq X$
$X \leq Y$ if X inherits from Y, $X \leq Z$ if $X \leq Y$ and $Y \leq Z$ — Transitive:

e.g. For an assign

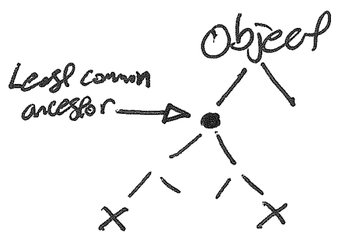
$$\frac{O(x) = T_0 \quad O \vdash e_1 : T_1 \quad T_1 \leq T_0}{O \vdash x \leftarrow e_1 : T_1}$$

[Attr-Init]

$$\frac{O_C(x) = T_0 \quad O_C \vdash e_1 : T_1 \quad T_1 \leq T_0}{O_C \vdash x : T_0 \leftarrow e_1;}$$

Least Upper Bound (lub(X, Y))

↳ This is the least common ancestor



$X \leq Z \wedge Y \leq Z$
where Z is an upper bound

For example with if-then-else expressions, what the expression might return is just the lub(T_1, T_2), the least common ancestor as we don't exactly know what the overall type might be

$$\frac{O \vdash e_0 : \text{Bool} \quad O \vdash e_1 : T_1 \quad O \vdash e_2 : T_2}{O \vdash \text{if } e_0 \text{ then } e_1 \text{ else } e_2 \text{ fi} : \text{lub}(T_1, T_2)}$$

◁ needs all of these

Type checking methods

↳ Two different name spaces/environments in Cool, one for methods and other for objects, these can both live in the same scope

e.g. in types rules this is reflected by mapping M for method signatures:

$$M(C, f) = (T_1, \ldots, T_n, T_{n+1})$$

C: class; f: method; $T_1, \ldots, T_n$: inputs into method; T_{n+1}: return type or result of function

in other words in class C we have a method f such that:

$$f(x_1 : T_1, \ldots, x_n : T_n) : T_{n+1}$$

All rules get passed down the method environment

↳ Only dispatch rules use M.

e.g. full logic:

$$O, M, C \vdash e : T$$

Implementing Type Checking

COOL type checking can be implemented in a single AST traversal (Environment is some Object)

↳ Recursive,

```
TypeCheck(Environment, e1 + e2) = {
    T1 = TypeCheck(Environment, e1);
    T2 = TypeCheck(Environment, e2);
    check T1 == T2 == Int; return Int; }
```

Lecture 10

Static vs. Dynamic Typing

- Dynamic Type: is class C used in the "new C" expression → Run-time notion
- Static Type: captures all dynamic types
  ↳ A Compile-time notion

Soudness Theorem:
$$dynamic_type(E) = static_type(E)$$

$$\forall E.\ dynamic_type(E) \leq static_type(E)$$

All operations on object of type C can also be used as $C' \leq C$

Subclasses only add attributes/methods

Can be redefined but with same type

Self Type

If a method returns "self" the capter can't accurately describe the return type using only the class name b/c 'self' might be an instance of a subclass

↑ In that case we can't differentiate b/w the two

Now, SELF_TYPE allows inc return type to change when inc gets inherited

(new Count).inc() : Count
↳ Type checker can now prove this is well typed

SELF_TYPE is a static type of not a class name
↳ Type checker keeps better track of types

Self-Type Operates
Self type depends on where it appears, e.g. $SELF_TYPE_C$ is the SELF_TYPE occurs in C

If E has type SELF_TYPE inside C then the object produced by E must have dynamic type C or subclass of C.

$T \leq SELF_TYPE_C$ ↛ Always false
↳ Fun bf $T \leq T'$

$lub(SELF_TYPE_C, T) = lub(C, T)$
↳ which is best we can do b/c $SELF_TYPE_C \leq C$

SELF_TYPE only appears where a type is expected
↳ But not allowed to be everywhere a type can appear.

[T can be SELF_TYPE when let x:T in ∈
↳ SELF_TYPE follows runtime type of self

example of the runtime error when m(x:T):T'{...}
↳ Adding SELF_TYPE changes everywhere in type checker where types are compared

$T_1 \leq T_0$ where T_1 is a subtype of T_0

for example when Id ← e_1 : T_1 when $O(Id) = T_0$ was first defined to have type T_0 before it gets defined to have type T_1 as it's a subtype of T_0

The type of the dispatch is the type of the dispatch expression
↳ ~~call~~ returns the same type as object called on, not the type in the method definition
↳ object for recursive method call.

A static dispatch e_0@T.f(...) : T_0, the result type becomes type of object called. Method preserves caller's type instead of becoming T.
↳ i.e. "return the caller's type."

So the new rules:

O, M, C ⊢ self : SELF_TYPE$_C$

O, M, C ⊢ new SELF_TYPE : SELF_TYPE$_C$

Self + new SELF_TYPE preserve the current object's type

takeaways

↳ ≤ and lub and extended operations can do a lot of the "work"

•SELF_TYPE can only be used in a few places

↳ It is a research idea adding more expressiveness to the type system ↳ Goal is to highlight that type checking can be very subtle.

Error Recovery

↳ Easier b/c AST already exists (we have structure)
?What to assign expressions w/o a real type? This influences types of all future expressions

↳ e.g. assign object (fine for project (safe as it's a subtype)

This is workable but cascades up higher consistently giving problems

No longer a tree however

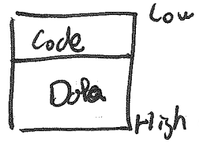
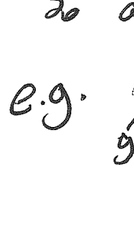
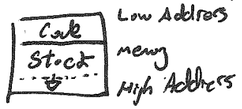
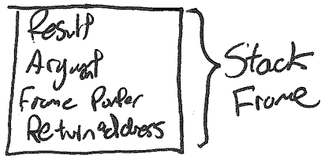
↳ e.g. No_type, error recovery and error checking

e.g. Define No_type ≤ C, all operations are defined for No_type

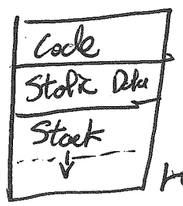
Lecture 11 - Runtime Organization

⟹ Managent of run-time resources which is the correspdee b/w struct of dynamic structures
(compile time / Run-time)

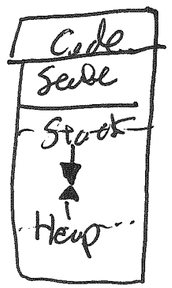
- Storage Organization

~~~

e.g. Program is init under control of OS
↳ OS allocates space, OS jumps to entry point

~~

Compiler is responsible for
- Generating code
- Orchestrating use of data area

e.g.



Activations include complications trying to solve
- Correctness
- Speed

Both problems at same time

Assumptions
1) Execution is sequential [Pointer moves one at a time / e.g. concurrency is exception]
2) Control returns to point after call

Invocation of procedure P is an activation of P

i.e. A running instance is an activation
So calling foo() means an activation of foo begins

Lifetime of P → From the moment the f(x) called until it finally returns
↳ All steps of execution

→ Lifetime is a dynamic (run-time) concept
→ Scope is a static concept (where it's visible)

So activation lifetimes can be written as a tree.

e.g.



g gets called in f and g and f are disjoint from each other.

Can also be represented with a stack (kinda of like stack pointers and frame pointers).



Activation Record ≡ Frame
↳ Holds information (primarily on stack) for state and execution to finish.



Located in address "array" memory

Will be able to find the frame pointer value as a simple offset

☆ At compile time, activation record layout has to be determined to generate code correctly in activation record → AR and code generator has to be designed together

Globals + Heap

Global can't be stored in AR
↳ Instead they get a fixed address where the fixed address gets "statically allocated"
~~~

Now with Global Variables:



↳ Heap contains all other data
↳ Managed by Malloc & free
Heap + Stack grow, so put them in opp. side of memory and make them grow to each other.



Alignment

Word aligned if it begins at a word boundary

↳ Most machines have alignment restrictions
↳ Performance penalties for poor alignment to use memory and not waste bytes.
Just add padding bytes.

Stack Machines

↳ Only Storage is a Stack

- Location of operands/result is not explicitly stated b/c it is always at the top of the stack
  ↳ Compare this to a register machine where you must specify the register location
  ↳ More compact program

n-register stack machine

Keeps top n locations of pure stack machine's stack in registers.

1-register stack machine → Accumulator

After evaluating e, the accumulator holds the value of e and the stack is unchanged

Expression evaluation preserves the stack

Lecture 12

↳ Introduction to code generation

- Generate code for a stack machine w/ accumulator
  ↳ we want to run resulting code on a real machine
    e.g. MIPS processor → RISC

Setup:
- Accumulator kept in $a0
- Stack is kept in memory
- Stack Pointer kept in $sp address

(32 bit registers)

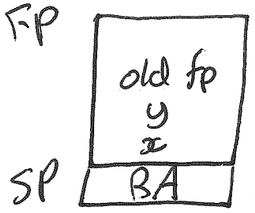
example stack-machine code for 7+5 in MIPS

- Now lets go to code generation for languages more complicated than above code stack-machine
  ↳ Define a recursive gen function, who result is the code gen for e. (cgen(e))
    ↳ Code generator
- example should how stack not a temp register, is very much needed to keep state
  cgen(3+4) ↝ e.g. means to generate the assembly code that evaluates this expression for expression e.

Code generation II

↳ function calls + function definitions depend on the layout of the AR

- No need for a control link b/c of $sp getting saved
  ↳ Pointer on current activation helps ↳ Lives in $fp frame pointer register



jal ← Jump and Link, saves the value of the frame pointer and the actual params in reverse order
label

e.g.

```
sw $fp 0($sp)      Store word
addiu $sp $sp -4   Decrement Stack Pointer
cgen(e_n)          ← Call/gen other expression body of function
:
```

- "Variable" of a function are just its parameters
  - which are all in the AR
  - pushed by the caller
  - But stack grows down
  - intermediate results are saved → variables are at a fixed offset. This is a problem
  - Use Frame Pointer instead

$cgen(x_i) = lw\ \$a0\ z(\$fp)$ where $z = 4*i$

Code gen can be done by recursive traversals of AST
- use stack machine for cool

Production Compilers are different:
- keep values in registers

Temporaries

- keep temporaries in the AR, code gen must assign a location in the AR for each temp
  - Fixed
  - Save and store w/o stack pointer manipulation

e.g. det fib(x) = if $x^1 = 1$ then 0 else
if $x^1 = 2$ then 1 else
1 fib(x-1) + fib(x-2)^1

So we get 4 temporaries
But we only need 2 temporaries
b/c temp #1: Save x
temp #2: Save fib(x-1)
we need to remember og x and the result of fib(x-1)

NT(e) = # of temps needed to evaluate e

e.g. NT($e_1 + e_2$)
- At least NT(e_1) and NT(e_2)+1 as we need to hold onto a temp while using e_2.
- Space for temps in e_1 can be reused for temps in e_2.

So... $NT(e_1 + e_2) = \max(NT(e_1), 1 + NT(e_2))$

So once the temporaries are found, we get $f(x_1, \ldots, x_n) = e$ where the AR has $2 + n + NT(e)$ elements

- Return Address
- Frame Pointer
- n arguments
- $NT(e)$ locations for intermediate results

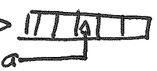
Better than saying these values in memory stack is easier.

Object Layout

OO Slogan: if B is a subclass of A, then an object of class B can be used wherever an object of class A is expected.

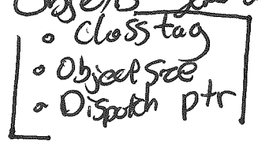
e.g. attribute "a" must be in the same place in each object

- Objects are laid out in contiguous memory

Self →  A, B, C

- Each attribute is at a fixed offset in the object

Objects look as:



→ Attributes in subsequent slots, all layout in contiguous memory

B can just be A with additional slots for extra attributes of B.

↳ Visually one finds the B attributes in the same place as the A attributes ↘ You get extra attribute

So same code would run (Stack/memory/ptr offsets etc. are the same)

$A < C < B$

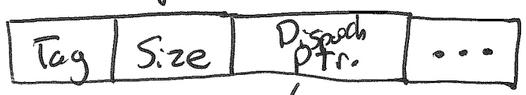
↳ "inherits symbol"

Every class has a fixed set of methods

- Dispatch table indexes these methods

Dispatch pointer in an object of class X points to the dispatch table for class X.

↳ Every method of X is assigned an offset at compile time.



↳ ↙ This can be shared which makes it convenient to share

Lecture 13

→ Semantics
  ↳ What happens when it is evaluated
  the "meaning" of an expression

↳ Definition of a programming language:
- tokens => lexical analysis
- grammar => syntax analysis
- typing rules => ...
- evaluation rules => code gen and optimization

↳ We have specified evaluation rules indirectly
- Compilation
- Evaluation

↳ But this isn't good enough
  ↳ There are tons of missing details.
e.g. Whether to use stack machine, or which one to
  ↳ But we don't want overly restrictive specification

— Operational Semantics ↳ Useful to specifying implementations
  ↓
  Describes program eval with execution rules ↳ Most useful for implementations
  → Denotational Semantics => Mapping goes to with Functions
  → Axiomatic Semantics ↳ For cool
    ↳ Described through logical formulae

Operational Semantics
↳ Context ⊢ e : v
  in a context (Special?) evaluates to some value v.

- Environment → Where in memory a variable is
- Store → What value is in the memory

Variable Environment maps variables to location

$E = [a : I_1, b : I_2]$, only those in scope get applied here
(a: variable, I_1: location)

Meanwhile Stores:

$S = [I_1 \to 5, I_2 \to 7]$, where
I_1: Location in Store # 5

$S' = S[12/I_1]$ defines S' s.t. $S'(I_1) = 12$

=> All objects are instances of some class

$X(a_1 = I_1, \ldots, a_n = I_n)$

X: class, a_i are the attributes, I_i is location where a_i is stored

Contexts: $O, E, S \vdash e : v, S'$
- O: current value of self
- E: variable environment
- S: current Store if another termination
- v: value
- S': New Store with new value in it

Cool Semantics

e.g.
$$\frac{O, E, S \vdash e_1 : v_1, S_1 \quad O, E, S_1 \vdash e_2 : v_2, S_2}{O, E, S \vdash e_1 + e_2 : v_1 + v_2, S_2}$$

Adding values

$[? \leftarrow 12]$, e.g. we store 12 into location ?

$I_{new} = newloc(S)$ ↳ Says I_{new} is a location not already used in S.

↳ newloc is like the memory allocation function

Semantics of new T ↳ Allocate a new Object
- ↳ Set attributes to default values
- ↳ Evaluate initializers and set resulting attribute values
- ↳ Return newly allocated object

Class $(A) = (a_1 : T_1 \leftarrow e_1, \ldots a_n : T_n \leftarrow e_n)$

- ↳ a_i: inherited ones
- ↳ T_i: types
- ↳ e_i: initializers

☆ Greatest ancestors first

↳ class $(C) = (a_1 \ldots, a_2 \ldots, b_1 \ldots, b_2 \ldots, c_1 \ldots, c_2 \ldots)$

↳ First 3-steps allocate the object
↳ Remaining steps init object

↳ Scope is which initializers are available
↳ Only attributes are in scope
↳ Init values of attributes are defaults
↳ If object, v is new object that is current object being initialized.

$impl(A, f) = (x_1, \ldots, x_n, e_{body})$

x_i is name of formal arguments
e_{body} is body of method

$impl(x, f)$

↑
dynamic type of v_0, we can use that to find the function to run.

E' environment for the parameters is rather different to the subsequent E which contains a very different E

E → supply found arguments and self's attributes

S → Like the caller's except with actual arguments bound to the locations allocated for formals

↳ Frame is implicit (we don't actually talk/worry about it)
↳ Semantics of static dispatch is similar

↳ Operational rules are very precise and detailed ↳ Nothing is left unspecified

Lecture 14

Intermediate code
↳ Language b/w source and target
↳ Provides intermediate lvl of abstraction

Source → IL → Target , e.g. llvm for intermediate

where x := y op z ← y and z are registers
x := op y ← or constants

One operation at a time
↳ Each sub expression/intermediate subexpression has a "name"
↳ Similar to assembly code generator

e.g. igen($e_1 + e_2$, t) =
igen(e_1, t_1)
igen(e_2, t_2)
t := $t_1 + t_2$

Optimization Overview
↳ Most complex part of compiler

So instead implement optimization on intermediate language

Idea: can say there is a basic block which is a maximal sequence of instructions
- No labels
- No jumps → So can't jump in or out of block

Only way is in at the beginning and out at the end

Control-flow graph
↳ A directed graph with basic blocks as nodes (just input and output fresh).

Optimization improves a resource utilization
↳ Execution time (most often, what we doing)
↳ Optimization should be the same here)
- Local
- Global Optimizations
- Inter-procedural optimizations [Apply across method boundaries]

Goal: max benefit for min cost

Local Optimizations
↳ Focuses on one block

e.g.
→ Some can be deleted
→ Some can be simplified e.g. x := x * 0 => x := 0
e.g. x := x * 8 => x := x << 3

Constant Folding → operations on constants can be computed at compile time
x := 2+2 => x := 4
↳ Could be different results with floating point numbers giving the precision centered on two machines (cross compilation)

↳ Remove unreachable basic blocks, makes it smaller and thus faster

Why does unreachable blocks occur?

↳ e.g. #define DEBUG 0 basic block to strip out unneeded code

↳ Or a library

Get intermediate code into Single assignment form

```
x := z+y          b := z+y
a := x      =>    a := b
x := 2*x          x := 2*b
```

→ Common subexpression operation
→ Copy propagation
- Performing one optimization enables another
↳ Does it repeatedly until why works.

Peephole Optimization

↳ Works on assembly code instead of IR or higher lvl.
↳ Many of basic block optimizations can be put as peephole optimizers

e.g. addiu $a $a i, addiu $a $a j
→ addiu $a $a i+j
↳ Apply repeatedly like other opt

* Program improvement

Lecture 15

Global Optimization, start with dataflow analysis
↳ These can be extended to an entire control-flow graph ↳ How do we know it's ok to propagate constants??

To replace x by k, every path to x, the last assignment to x is x:=k

"All paths" include loops + branches of conditionals

↳ So requires analysis of entire control-flow graph which is the global dataflow analysis.
↳ Rather be safe, instead of being wrong and causing havoc. = "don't know"

Constant Propagation

↳ On every path to use of x, the last assignment is x:=k
Perform at any point where ** holds

Value	Interpretation
~~I~~	~~this statement never~~ executes
C	X = constant C, it has a very specific value, constant C
T	X is not a constant, we don't know, could have any value

"top"

- Complex program is expressed as a combo of simple rules relating chge in info b/w adj. statements

↳ Idea is to "push" or "transfer" info from one statement to the next

$C(x, s, in)$ = value of x bf s

$C(x, s, out)$ = value of x after s

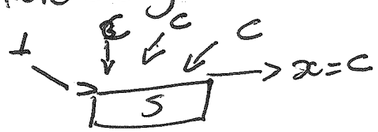
transfer function, transfers info from one statement to another

↳ Let s have immediate predecessor statements $p_1, \dots, p_n$

if $C(p_i, x, out) = T$ for any i, then $C(s, x, in) = T$

B/c one T means any other input is unknown so goes to unknown as well

Rule 3 e.g.



b/c ⊥ shows we never reach

we got 8 rules

Algorithm to assign

1) ↳ for every entry s, set $C(s, x, in) = T$

2) ↳ set $C(s, x, in) = C(s, x, out) = \perp$

3) ↳ Repeat until all points satisfy rules 1-8

(all in slides but pretty self explanatory)

Analysis of Loops

- B/c of cycles, all points must have values at all times. ↳ Assign an initial value to break the cycles

↳ ⊥ is the initial value

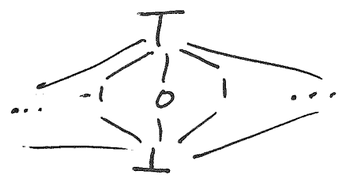
So you check if its consistent in every point then looping

Orderings

We can simplify presentation of analysis by ordering the values

$\perp < c < T$

These are abstract values



most

Equal sorta

least

T is the greatest value, $\perp$ is the least

↳ lub is least-upper bound in ordering

take the greatest value

bigger than both

e.g. $lub(T, \perp) = T$, $lub(1, 2) = T$

"Repeat until nothing changes ↳ doesn't mean nothing changes

- Values start as $\perp$ and only increase

$C(s, x, in/out)$ only changes twice

Number of steps = # of $C(\dots) * 2$ =

# of program statements * 4

Liveness Analysis

e.g. if X is dead or alive (may be used in future)

live = may be used in the future

e.g. $X := 3$ is dead as it's never used

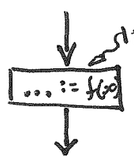
X is live at S if...

→ there is S' that uses X
→ There is a path from S to S'
→ That path has no interveny assignment to X

☆ dead statement can be deleted, but removal to know that.

Liveness is just information transferred b/w adjacent statements ▷ Simpler than constant propagation

▷ Liveness is simpler than constant propagation as it's a boolean property



L(S,x,in) = true if S refers to x on the rhs.

e.g. if e does not refer to x, not live!

else L(S,x,in) = L(S,x,out) if S does not refer to x

↳ Value can change only once so termination is guaranteed

↳ From false to true

ConstantPropagation => Forward analysis (inputs to outputs)
Liveness => Backward Analysis

↳ Must follow "local rules" b/w adj program points

Lecture 16

↳ Register Allocation
- Intermediate code uses unlimited temporaries
  - Simplifies code gen and optimizer
  - Complicates final transfer to assembly

→ IR can have too many temporaries

↳ Rewrite intermediate code to use no more temporaries than there are machine regs

temp many → reg 1 } > Assign multiple temps to each reg
- Don't change the program's behaviour (ofc)

Graph coloring: a reg alloc algorithm

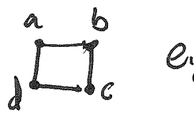
[if t_1 + t_2 are live at same time, they can't share a register]

↳ So find the two values that are live and make sure they don't overlap

↳ Add a node for each temporary

- Register Interference Graph (RIG)

↳ Two temps can be alloc'd to same reg if there is no edge connecting them



e.g. • b+c can't be in same reg
• b+d could be in same reg
} > Two temps can be allocated w/o edge

Exactly info needed to choose/coalesce legal register assignments ↳ Also gives global view

Graph Coloring

aka registers

Assign colors to nodes, e.g. k-colorable has k or fewer colors (CIS1600)

k = # of machine registers

How to compute graph colory

How to compute graph colory?

↳ Very hard, no efficient algorithms are known.

algo: to compute graph coloring:

1) → Pick node with few neighbors, put it on stack and remove it from RIG
2) → Assign colors with least used colored

Spilling → what to do when we can't find a coloring??

↳ we can't hold all values in registers, so we spill the values to memory.

So...

→ Pick a node as a candidate to spill

4 neighbors of f we use less than will live in very 3 colors e.g. optimistic coloring

↳ Allocate a memory location for f.

e.g. call it fa, f := load fa

So you end up adding multiple store, load f1 all f's around across graph.

↳ New liveness info is almost like before, only that f is split into 3 temporaries

fi is live only b/w a fi:=load fa and next instruction, b/w a store fi, fa and preceding instr.

↳ Spilling reduces live range of f
- Reduces interferes
- Fewer RIG neighbors

→ Additional spills might be required if "coloring" is found
→ Deciding what to spill
→ Spill temporaries w/ most conflicts
↳ Spill heuristic that hurts performance the least

Lower cost = better candidate to spill
↳ Loop nesting can substantially

Register allocation is "must have"
↳ Intermediate code has too many temps
↳ Makes big diff. in performance
Register allocation is more captured for CISC machines (more complex)

Managing Code

How compilers can merge code
↳ typically very good at managing registers

→ Loop invariant optimization can be done based on cache's structure in memory so that they refresh

Lecture 17

Automatic Memory Management ↦ Garbage collection

- Forgetting to free unused memory
- Dereferencing a dangling pointer
- Overwriting parts of a data structure by accident
- ☆ Storage bugs can be hard to find
↳ Visible effect far away

Goal is complete automatic memory management
↳ Became mainstream w/ popularity of Java

→ When an object is created, unused space is automatically allocated
↳ We get, after a while, no more unused space
↳ So we need to identify space that is occupied by obj to never be used again
— Can be freed to be reused later

So how do we know object "will never be used again"?
↳ A program can only use objects it can find

let x: A ← new A in {x←y; ...}

Object x is reachable iff
- A reg contains a ptr* to x
↳ Seeing by registers and then following all the pointers

↳ An unreachable object can never be used
• Such objects are garbage

e.g. After $x \leftarrow y$ the obj A has no reference pointing to it so it becomes unreachable and can be collected. while B is still reachable thru x

Reachability is an approximation. Say that

$x \leftarrow$ new A "doesn't happen" or if we have y not point to smt new

Coolc uses an accumulator

↳ Stack frame contains pointers

Tracing from acc and Stack

↳ Roots

eg Note B+D are unreachable from acc or Stack

1) Allocate space as needed for new object
2) When space runs out
   - Compute what obj might be used again
   - Free space used by obj not found in (a)

Mark and Sweep

- Mark: traces reachable objects
- Sweep: Collects garbage objects

Reserve for mem manager, where mark bit kept set to zero initially

For the sweep, we scan the heap and look for objects w/ mark bit = 0. These get added to the free list. Then

Serious problem with mark phase

↳ Invoked when we are out of space

e.g. pointer reversal

reversed to point to parent

Aux data stored in the object themselves

Free list gets stored in free objects

- Space for new object gets allocated from the free list
- Mark and Sweep can fragment the memory ... using adjacent blocks might be broken and

Advantage: objects are not moved during GC

Stop and Copy

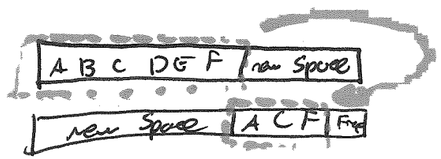
heap pointer → Old Space | New Space ← just for GC

- Heap ptr* points to next free word in the old space
- Allocation just advances heap pointer

GC starts when old space is full

Copy "pointer graph" over to the new space.

↳ Also known as the old Space from the left

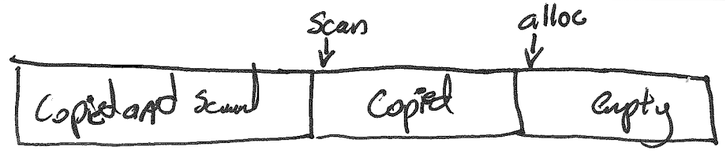


Copy over here to the new space and repeat

① Find all reachable objects then copy it to the new space

② Store a forwarding ptr* to the new copy

↳ So we know that ptr was alr. copied



Field ptr* have been followed
Ptr* have not been followed yet

Object in new space point to objects in old space and get copied over as a result → Extra work

↳ If we still have objects b/w Scan & alloc ptr there still is some work to do in the Scan phase

- Similarly to mark + sweep, we must know how big an obj is when we Scan if true gotta know where ptr* are inside obj
- We must also copy any obj pointed by stacks

Stop and copy is the fastest GC technique

Allocation cheap
collection is cheap (doesn't touch garbage) → only reachable objects
Some languages do not allow copying, e.g. C or C++

Conservative Collection

GC requires us to find all reachable objects
↳ Find all pointers in an object
for C or C++ it can be impossible to identify where objects in memory ↳ Can't guarantee we know where all ptr* are

OK to be conservative

↳ If a memory word looks like a ptr* it is considered a pointer. e.g. must be aligned ↳ must also be a valid address

↳ But we still can't update ptr* to them as we can't move objects

Reference counting, collect an obj when there are no more pointers to it

↳ Store in each obj the # of ptr* to that obj

So when we assign a value we update the reference count telling us the # of ptr* in that object

Disadvantage ↳ Can't do circular structure
↳ Changing reference counts is very slow (multiple updates per assignments)

↳ Automatic memory managed prevents storage bugs

- Reduces programmer control ↳ Memory leaks possible

↳ Pauses can be problematic in real-time apps

More advanced:
- conservative
- generational
- Real time
- parallel

↳ Even Java can have memory leaks

Lecture 18

Applying concepts to Java, analyzing it

Java - COOL + more on steroids

↳ Top dogs of an Internet language (killer app)

↳ Exceptions, interfaces, threads etc.

Java Arrays

- Having multiple aliases to updatable locations w/ different types is unsound aliasing

↳ Standard solution: Disallow subtyping through arrays by checking each array assignment at runtime for type soundness

↳ Is type of object assigned compatible with the type of the array.

- Adds overhead though on array operations

↳ Primitive types are unaffected, b/c they are not classes.

↳ Types.

Java exceptions

↳ throw up till hits try catch (∞)

$T(v)$ = exception thrown with value v

$$\frac{E \vdash e : v}{E \vdash \text{throw } e : T(v)}$$ or more specifically

$$\frac{E \vdash e_1 : T(v_1)}{E \vdash e_1 + e_2 : T(v_1)}$$

When we encounter a try

→ Mark current location in the stack & have it propagate out ↳ Unwind stack ↳ execute catch

→ execute correctly catch ↳ more complex techniques to reduce try + throw

Unwind stack on first try, then

This is key

☆ An uncaught exception thrown during object finalization gets ignored by JVM and doesn't propagate to the rest of program.

↳ Methods must declare types of exceptions they may raise

e.g. type rules ↔ throw must be of obj Exception

↳ The types associated with exceptions are the corresponding types

Java interfaces

↳ Relationships b/w classes that don't have inheritance

implements interface / interfaces play role of multiple inheritance in C++ b/c classes can implement multiple interfaces

e.g. class X implements A, B, C {...}

e.g. graduate student may be both University employee & student

Graduate student can implement both university student & employee

Methods in class$ implies interfaces need not be at fixed offsets.

No known total order otherwise gotta exed it further
↳ b/c method don't live at fixed offsets

One approach

- Each class has a lookup table: method names → code
- Hash method names for faster lookup

Java Coercions

→ Primitive types can be converted (aka coerced) in some contexts

e.g. 1 + 2.0, the 1 gets "idol to a float 1.0

↳ Coercion is only a primitive function the compiler inserts ↳ Compiler notices you "left out" a function call & inserts it in.

- Widening (int → float) ← can be implicit "always succeed"
- Narrowing (float → int, downcasts)
  ↳ Must be explicit, e.g. 2.5 (float) → (int)

Bool is only exception to coercion/cast
↳ Can be ehh cuz gives unpleasantness results maybe.

Threads
↳ Has its own PC + Stack

Thread objects have start + stop methods

with a scheduler
- Pick thread
- Execute 1 steptemp } => Synchronization puts a lock on the obj

Synchronized(x) {e}
"this" is locked
Lock x, evaluate e, unlock x } => How to control interleaving

Writes must be synchronized with the "synchronized" keyword for both threads not just one

This allows it S.t. all of to() finishes before all of fro() or vice versa.

- Writes of values are atomic
- Violated for doubles though
- Java concurrency semantics are difficult to understand in detail

Other Java Topics

↳ Classes loaded at run time
- Type checking takes place at compile time
- Bytecode verification takes place at run time
  ↳ Type checking of Bytecode

- Initialization in Java is complex → COOL ++
  ↳ Correctly captured by counting
When Symbol in class is first used
  ↳ Not when class is loaded
Class object → object representing a class

→ Wait on lock if another thread has locked it
↳ else release lock at return

With N locks, we have expected N^2 locks. Pairwise interacts, e.g. N^2 locks interactions ↔ Big locks ful systems are hard to understand. (includes programming languages)
- Stay sceptic of garbage collector

Midterm #1 - Spring 2024

1. We start with $S_0 = \{q_0\}$

as there are no ε transitions would require more moves to get out

$S_0 \xrightarrow{a,b} S_1$, we go to q_1 or q_2

$S_1 = \{q_1, q_2\}$

We see that from q_1 we can get to q_3 (the accepting state) by taking

a, b or c or c, b

S_0 S_3 can be arrived at from S_1 taking ε, b while S_5 can be arrived at from S_1 with c and S_6 can be arrived at from c, b

Finally with $q_0 \xrightarrow{b} q_1 \xrightarrow{\varepsilon} q_2 \xrightarrow{b} q_3 \xrightarrow{b} q_3$ we can arrive at the only other S_7 in similar light, $s_0 \xrightarrow{b} S_1 \xrightarrow{\varepsilon} S_3 \xrightarrow{b} S_7$

— Are only accepting states.

2.a) E = lowest precedence (^)
T = middle precedence (&)
F = highest precedence (~)

So...

E -> E ^ T | T
T -> T & F | F
F -> ~F | id

b) we can write

A_SJC -> SJC -> A_LAX
A_SJC -> SJC -> A_DFW
A_LAX -> LAX -> A_SJC
A_LAX -> LAX -> A_DFW
A_LAX -> LAX -> A_ORD
A_DFW -> DFW -> A_SJC
A_DFW -> DFW -> A_LAX
A_DFW -> DFW -> A_ORD
A_DFW -> DFW -> A_SFO
A_ORD -> ORD -> A_LAX
A_ORD -> ORD -> A_DFW
A_ORD -> ORD -> A_SFO
A_SFO -> SFO -> A_DFW
A_SFO -> SFO -> A_ORD
A_SFO -> SFO

The A prefix explains which airport we are currently at

3a) First (S) = {a}

First (E) = {c, ε}

First (Q) = {a, ε}

b) Follow (S) = {$}

Follow (E) = {$} ← These are subsets of S

Follow (Q) = {$}

c) No b/c S->aE and S->abQ both start with a terminal a, so a one token lookahead can't decide which production to use

d) Second (S) = {c, b, ε} find first of a:

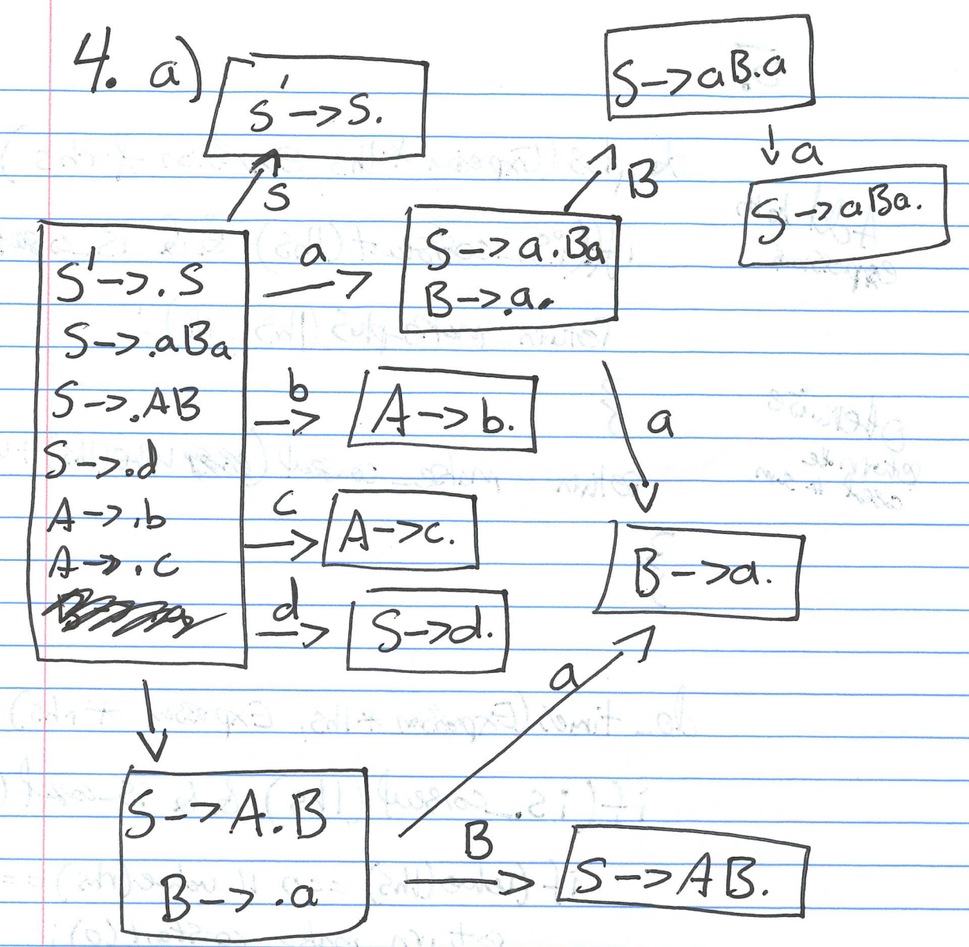
Second (E) = {a, b, ε} First(a) = {c, b, ε}

Second (Q) = {c, [illegible] ε}

e)

t_1	a				b				c				$
t_2	a	b	c	$	a	b	c	$	a	b	c	$	
S	abQ		dE	dE									
E									cQ	cb	cQ		ε
Q			dE	dE									ε

4. a)



b) If any no-lookahead state has conflict, not LR(0). We see that there are no reduce-reduce or shift-reduce conflicts, resolved

c) Yes valid same reasoning as above (b).

5.

Add both expressions

Otherwise returning the const # sum

```
do_plus(Expression *lhs, Expression *rhs){
  if((is_constant(lhs) && is_const(rhs))){
    return make_plus(lhs, rhs);
  }
  return make_const(value(lhs)+value(rhs));
}
```

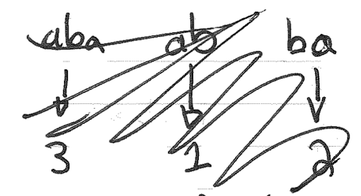
```
do_times(Expression *lhs, Expression *rhs){
  if(is_const(lhs) && is_const(rhs)){
    if(value(lhs) == 0 || value(rhs) == 0){
      return make_constant(0);
    }
    return make_const(value(lhs)*value(rhs));
  }
  return make_times(lhs, rhs);
}
```

Midterm 2 - Spring 2025

1) a) B/c it must accept the start state (ε) and then the state after having read "a" which means it must have at least two states

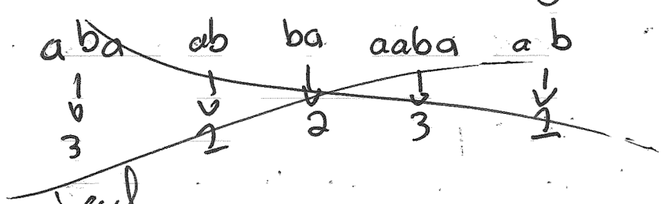
b) You make some new accepting state and then add ε-transition to said new accepting state

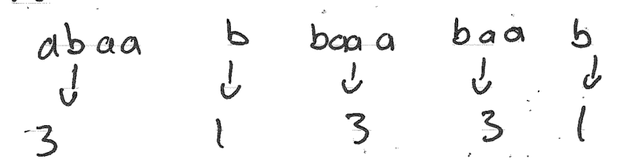
2.



b/c * means zero or more occurs with ε included

we also want the one ending in aa





3)

EPSILON

$$nullable = true
$$ beginnings = {}

CHAR

$$nullable = false
$$ beginnings = {$1.val}

~~expr '(' expr ')'~~

expr expr
if($1nullable)
$$ beginnings = $1beginnings
else
$$ beginnings = $1beginnings ∪ $2beginnings
$$nullable = $1nullable && $2nullable

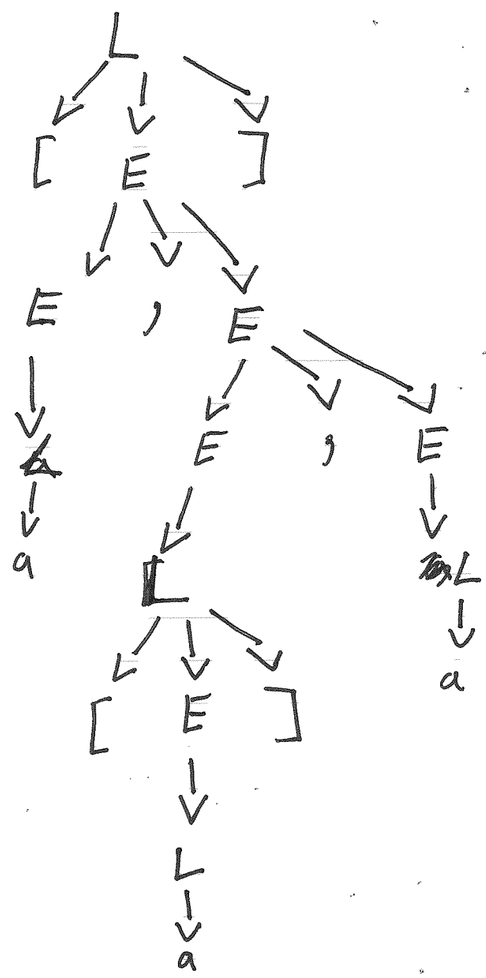
expr '+' expr
$$ nullable = $1nullable || $3nullable
$$ beginning = $1beginnings ∪ $3 beginnings

expr '*'
$$nullable = true
$$beginnings = $1 beginnings

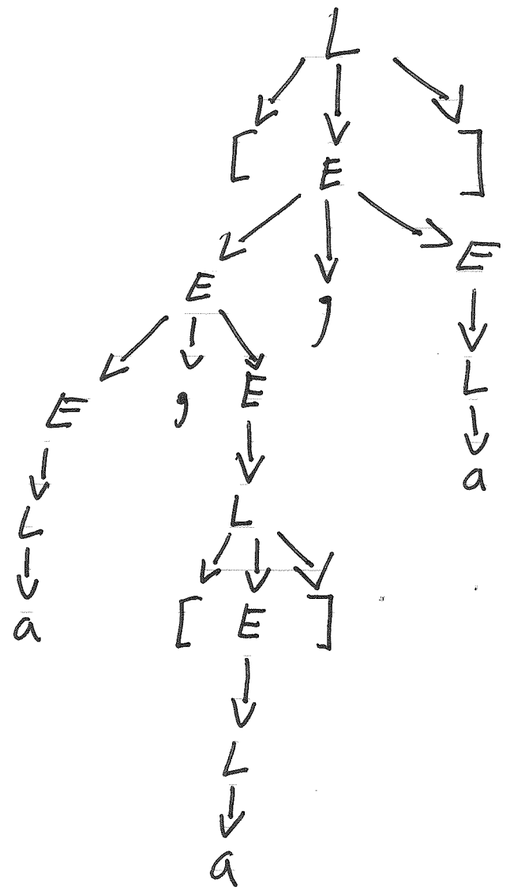
'(' expr ')'

$$nullable = $2nullable
$$ beginnings = $2 beginnings

4) a)



We can draw a second parse tree going to "the left" instead of to "the right"



b) First (L) = {[, a}, First (E) = {ε, ,, [, a}
Follow (E) = {,,], $}
Follow (L) = {,,], $}

c) We have First(E,E) = First (E) = {[, a, ε}
and First (L) = {[, a} so LL(1) which sees "[" or "a" doesn't know which one to choose.

d) L -> [T] | a
T -> T, E | E
E -> L | ε

L -> [E] | a
E -> L T | T
T -> , E T | ε

5) Really off State 0, we can see that
A -> xA
A -> y

Midterm 2026

1. ababa ab bb aba

↓ ↓ ↓ ↓

4 1 3 2

b/c there are less substrings required, just one

2. a) Yes it is b/c P and Q are interchangable. we can replace both P and Q back to back and get same e.g.

P a P b and P a P b

↓ ↓ ↓ ↓ ↓ ↓

a a Q a

↓

P

↓

a

which are two dif trees but output is the same.

b) four letters with _a_b and _b_a etc.

c) L(G) is regular b/c it is a finite language as it generates the four character strings. Thus it's regular.

as seen above

3) Node* handle_not(Node* e){

if(e.getType() == And){

return make_or(make_not(e.getLhs()), make_not(e.getRhs()));

}else if(e.getType() == Or){

return make_and(make_not(e.getLhs()), make_not(e.getRHS()));

}else if(e.getType() == Not){

return e.getOpnd();

}else if(e.getType() == Var){

return make_not(e);

}

}

4) a) Follow(S) = {$} b/c S is Start and nothing else in the grammar can appear after S

we can also see that A, b = ε and we know for LL(1) that if A -> ε put all ε-rules into the table. So...

Follow(A) = {b}

4. b) Ready from the table for A we see:

aA, ε, c

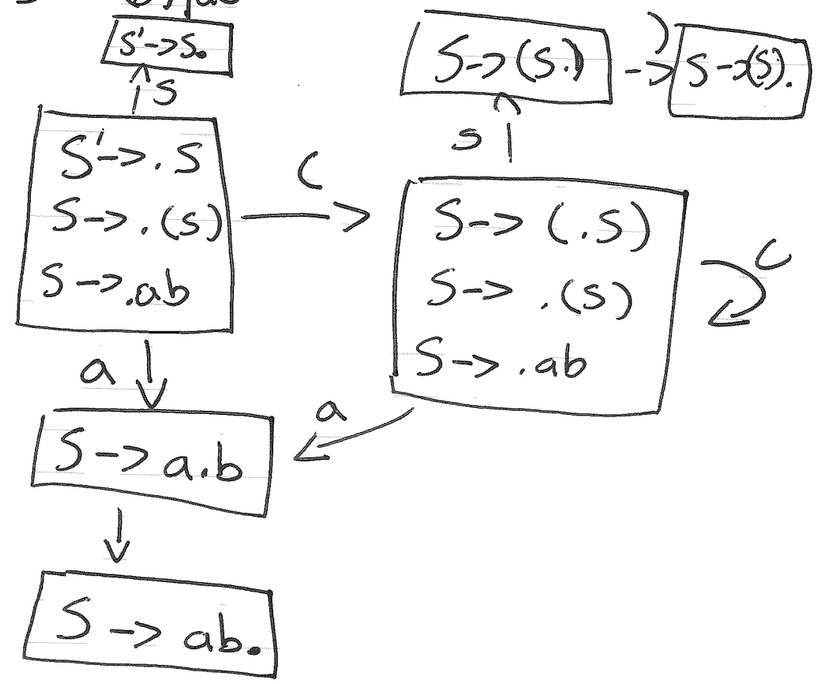
c) we see that $[S, c] = Ab$ b/c $c \in FIRST(Ab)$, so Ab goes in c column.

d) we see that $[S, b] = Ab$

5a) Taken as a given as no DFA parsing table is given in this midterm-2026

$S \rightarrow (S) \mid ab$

b)



Final 2024

1a) We know that $e_0.f(e_1,\dots,e_n)$ tells us that we call f on e_0 with $e_1,\dots,e_n$ as args

The key is that we ~~evaluate~~ Eck e_0 first

$$S_0,\ S_1,\ E \vdash e_0 \to V_0,\ S_2$$
$$V_0' = X(a_1 = la_1, \dots, a_m = la_m$$

⋮ Rest of the operand semantics

$$S_0, S_1, E \vdash e_0?.f(e_1,\dots,e_n) \mapsto V_{n+1}, S_{n+1}$$

b) Simpler b/c the entire expression returns void when the target object becomes void

$$S_0,\ S_1,\ E \vdash e_0 \to \text{void},\ S_2$$

$$S_0,\ S_1,\ E \vdash e_0?.f(e_1,\dots,e_n) \to \text{void}, S_2$$

#2. x is originally at 8($sp) then he adding 8 moves it down by 8, only 16($sp) for the first

In a similar logic, we can see that

4 ($sp)
12 ($sp)
4 ($sp)

Then filly at for label 0, 2, at 2 we get

4 ($sp)
8 ($sp)
4 ($sp)
4 ($sp)

#3. a)

Array

$C \vdash e_1 : T_1$
$C \vdash e_2 : T_2$
$\vdots$
$C \vdash e_n : T_n$
$T = lub(T_1, T_2, \ldots, T_n)$

$C \vdash [e_1, e_2, \ldots, e_n] : [T]$

Least upper bound, the closest shared parent

Array-Index

$C \vdash e_1 : [T_1]$
$C \vdash e_2 : Int$

$C \vdash e_1[e_2] : T_1$

Array then first indexed

[Array-Concat]

$C \vdash e_1 : [T_1]$
$C \vdash e_2 : [T_2]$
$T = lub(T_1, T_2)$

$C \vdash e_1 + e_2 : [T]$

b) No, even if chess ≤ boardGame, allow [chess] ≤ [BoardGame] would let someone treat [chess] array as a [BoardGame] array & insert a non-chess BoardGame obj. Then, once the original [chess] array gets read the cmpler will expect every element to be chess, but one element could just be BoardGame which breaks type safety

#4.

a)

1. a=1
2. b=1
3. c=1
4. e=a+b
5. d=1
6. d=c+e
7. d=d+e

Boards create the variable interferences

(working backwards from e being live at exit)

b) b/c of a, b, c being connected, we need to have three colors there, so K=3

Fewest neighbors + first in alphabetical order: d

c
b
a
e
d

first

c) r_1 : a=1
r_2 : b=1
r_3 : c=1
r_1 : e= a+b

r_2 is free

r_2 : d = 1

r_2 : d=c+e

r_2 : d=d+e

All registers now (at end) can be freed except r_2

5. BB1: 1-3
a) BB2: 6-14
BB3: 15-16
BB4: 4-5

b) (BB1, BB4)
(BB4, BB2)
(BB4, BB3)
(BB2, BB4)

c) We can see that c=0 then c=c+1 which violates the SSA rule/property as we are assigning the variable twice

d) We can elimate f=e line 7 as it gets used again in line 11 and just use e instead of defining a new variable e

we can also move 13 higher up to replace line 8. B/c no other BB all use i but is c is already incrubed we can replace it with e

Also g, h, and j serve no purpose in any future block here, so those can be eliminated.

Final 2025

1.a) Student is a subtype of Person. I.e. anywhere Student gets used so can Person.

① Student->Student ≤ Student->Person

Super if it can accept the broader input type Person and return the narrow type Student

② Person -> Student ≤ Student -> Student
combing ① and ②

③ Person -> Student ≤ Student -> Person

④ Person -> Person ≤ Student -> Person
combing ③ and ④

⑤ Person -> Student ≤ Person -> Person

b) For A must be broader than A' and B must be narrower than B' b/c of

$A' \leq A$

$B \leq B'$

Question 2

a)

```
if (t1 ≠ 0) goto (start);
```

start: expression 2

Execute expression 2 if $t1 \neq 0$

b)

$$\frac{C \vdash e_1 : T \qquad C \vdash e_2 : Int}{C \vdash \text{do } e_1 \text{ while } e_2 : T}$$

checks it now as a do-while loop

c)

$$\frac{C \vdash e_1 : T \qquad C \vdash e_2 = 0}{C \vdash e_1 \text{ while } e_2 : T}$$

When the integer is zero, we go to the false case

Giving

$$\frac{C \vdash e_1 : T \qquad C \vdash e_2 : T \qquad C \vdash \text{do } e_1 \text{ while } e_2 : T}{C \vdash \text{do } e_1 \text{ while } e_2 : T}$$

when we execute the true case, the integer is not zero

#3.a) B/c if it's not in SSA form, then we can't replace replace those variables/values with the previously defined values as the assignment might change said value.

```
a = 0
b = a+1
a = 1
c = a+1
```

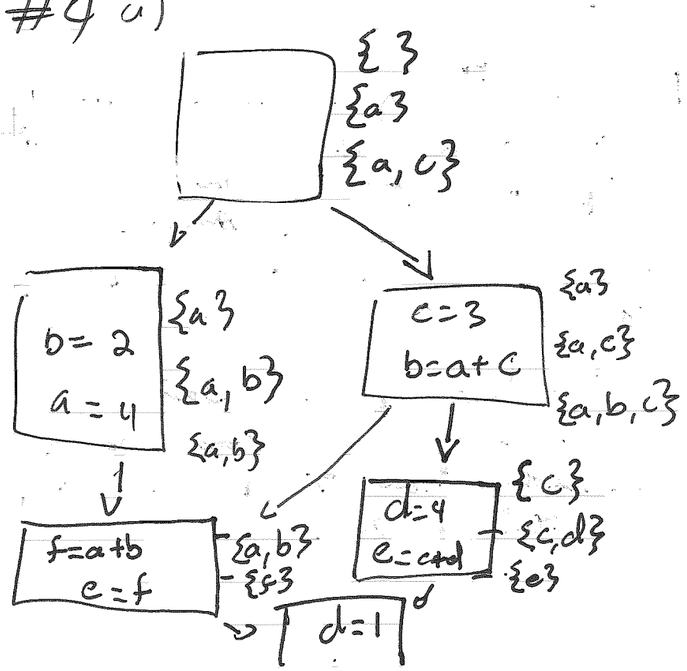
So we can't replace a+1 with b as the value of a changes

b)

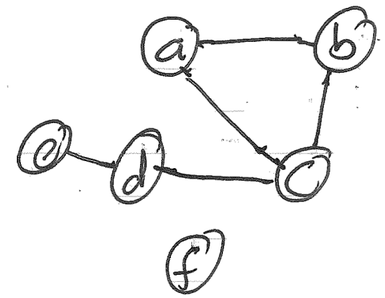
```
x = y + 0
z = x
return z
```

we can see that folding it first turns the expression into a simple copy then propagating removes it completely after.

#4 a)



b)



c)

r_1 : a, f, e

r_2 : c

r_3 : b, d

#5 a) When there are a few objects alive, stop+copy touches and copies only live obj while mark-and-sweep scans entire heap to find all free unreachable objects

b) When heap memory is almost full. B/c need to allocate new memory (e.g. disk) To temporary copy reachable mem before free everything else seems very costly rather than doing a single scan after the marks

c) If low pause time matters, e.g. mobile apps where can only frees objects immediately instead of stopping the whole program for a full heap scan.

#6.

We can add some inheritance relations to get

```
class A {
  x: Int <- 1
  h() : Int {
    0
  }
}
```

```
Class B inherits A {
  y: Int <- 2;
}
```

```
class C inherits B {
  f() : Int {
    0
  }
}
```

Final 2026

a) $O, M, C \vdash e_1 : Int$
$O, M, C \vdash e_2 : Int$
$O, M, C \vdash e_3 : T$

Grammar Production

$O, M, C \vdash$ for ID <- e_1 to e_2 loop e_3 pool : Object

b) Initializing the for-loop

$S_0, S_1, E \vdash e_1 \rightarrow Int(i), S_2$
$l = newloc(S_2)$
$S_3 = S_2[Int(i)/l]$
$S_0, S_3, E \vdash forloop(ID, e_2, e_3) \rightarrow void, S_4$

$S_0, S_1, E \vdash$ for ID <- e_1 to e_2 loop e_3 pool $\rightarrow$ void, S_4

Continue-loop

$S_1(E(ID)) = Int(i)$
$S_0, S_1, E \vdash e_2 \rightarrow Int(j), S_2$
$i \leq j$
$S_0, S_2, E \vdash e_3 \rightarrow v, S_3$
$S_4 = S_3[Int(i+1)/E(ID)]$
$S_0, S_4, E \vdash forloop(ID, e_2, e_3) \rightarrow Void, S_5$

$S_0, S_1, E \vdash forloop(ID, e_2, e_3) \rightarrow void, S_5$

Stop-loop

$S_1(E(ID)) = Int(i)$
$S_0, S_1, E \vdash e_2 \rightarrow Int(j), S_2$
$i > j$

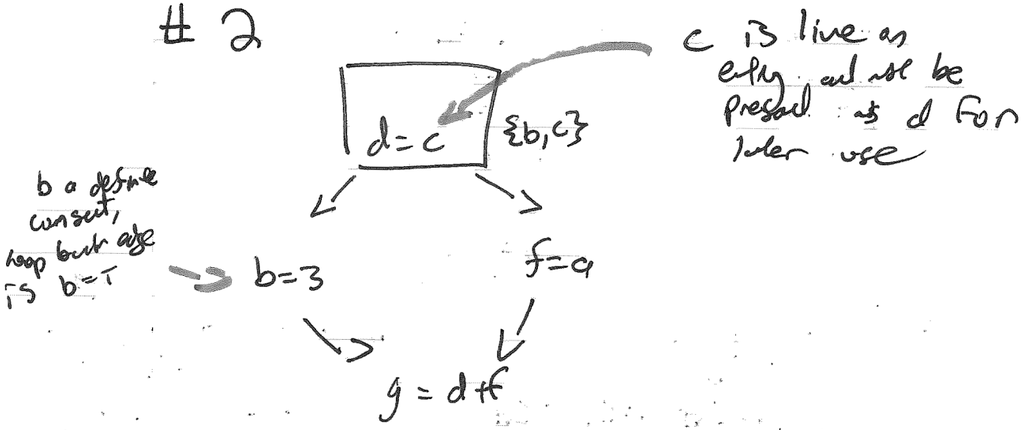
$S_0, S_2, E \vdash forloop(ID, e_2, e_3) \rightarrow Void, S_2$

c) cgen(for ID <- e_1 to e_2 loop e_3 pool) =
cgen(e_1)
push($a0)
bind ID -> 4($sp)
L:
lw $t1 4($sp)
lw $t2 12($a0)

cgen(e_2)
lw($t2 12($a0)
bgt $t1 $t2 Done
cgen(e_3)
lw $a0 4($sp)
add_int_const($a0, 1)
sw $a0 4($sp)

Done:
pop() Pop/remove temporary

2



#3

```
a = b + c
~~d = a + b~~
d = a + b
```

we can see here that before a=b+c, variable b and c are live, making the b — c edge before the second line where the a—b edge with d then getting isolated as it's only live after the last assignment.

#4

a)
```
a=new
b=new
c=new
d=new
e=new
a.a=e
a.b=b
a.d=d
b=null
c=null
d=null
e=null
```

b)
```
a=new
b=new
c=new
d=new
c=null
```

c)
```
a=new
b=new
c=new
b.a=c
c.a=b
b=null
c=null
```

#5
```
class Main {
  x: Int;
  f(y:Int, z:Int):Int {
    if 0<y then
      x<- x+z
    else
      0
    fi
  };
};
```

B/c the assembly first 0<y then is the true branch copies and stores x+z back into the object x while the false branch leaves 0